

Day 8 — Live coding: sostituire le bugie con Python

Il Day 7 si è chiuso con due bugie confessate: un saldo inventato da una riga `set_slots`, e un "il tuo appuntamento è prenotato" che non prenotava nulla. Oggi paghiamo quel debito. Python entra nel progetto attraverso le **custom action** — e ne usiamo tre per incontrare i tre volti dell'SDK: cosa un'action *restituisce* (event), cosa *legge* (il tracker), e cosa *invia* (il dispatcher). Lungo il percorso mettiamo in esecuzione un'API mock di core banking, colleghiamo il codice dell'action in due modi diversi, e mettiamo in scena sia un fallimento progettato sia uno non gestito.

Il filo conduttore della giornata: **la logica nei flow, il lavoro nelle action**. Un'action recupera dei fatti e riferisce se il recupero è andato a buon fine; il flow decide cosa l'assistente dice successivamente.

Lo stato finale è consegnato in questa cartella: `lumera-assistant/` (il progetto così come questo tutorial lo lascia), `lumera-fastapi-server/` (la banca mock), e un `Makefile` che esegue i processi.

REQUIREMENTS - il progetto Banca Lumera esattamente come lo ha lasciato il Day 7 (oppure copia la `lumera-assistant/` di questa cartella ed elimina i tre file `actions/*.py` per procedere passo passo) - `RASA_LICENSE` e `OPENAI_API_KEY` esportate nella tua shell - due variabili in più, usate dall'integrazione (il `Makefile` le imposta per i processi che avvia; esportale tu stesso in qualunque terminale dove esegui `rasa` direttamente):

```
export CORE_BANKING_URL=http://localhost:8000
export CORE_BANKING_TOKEN=test_token
```

- le dipendenze Python dell'assistente (`rasa-pro` e `requests`) — nessun passo manuale: `make setup` (§2) costruisce il venv per te, qualunque sia il punto di partenza scelto sopra

1. Le action erano in esecuzione fin dall'inizio

Prima di scriverne una, nota che il progetto "senza Python" di ieri era già pieno di action. Ogni turno termina in `action_listen` (attende l'input). Ogni sessione si apre con `action_session_start`. Ogni risposta che abbiamo innescato da uno step di un flow (- `action: utter_current_balance`) è passata attraverso il macchinario delle action, e i pattern integrati che hanno riparato le nostre conversazioni hanno eseguito action proprie. Ciò che il Day 8 aggiunge non è il concetto — è *il tuo codice* che si unisce a un sistema di action già esistente.

Una custom action è una classe Python con due obblighi:

```
class ActionSomething(Action):
    def name(self) -> Text:          # how YAML refers to it
        return "action_something"

    def run(self, dispatcher, tracker, domain) -> List[Dict]:
        ...                          # read state, do work
        return []                   # events that change the conversation
```

La firma di `run` è l'intera API, e le tre action di oggi si appoggiano ciascuna a una parte diversa di essa:

Parametro	Cos'è	La vetrina di oggi
<code>tracker</code>	vista in sola lettura della conversazione: slot, sender id, ultimo messaggio	<code>action_confirm_appointment</code>
<code>dispatcher</code>	il canale di output: testo, risposte, bottoni, JSON personalizzato	<code>action_send_statement_link</code>
valore di ritorno	event — fatti che rientrano nella conversazione, es. <code>SlotSet</code>	<code>action_fetch_balance</code>

(Un altro trucco per i giorni successivi: restituire una classe il cui `name()` corrisponde a un'action di default — per esempio `action_session_start` — sostituisce quel default con la tua versione. L'override-by-name è il modo in cui si personalizza la logica di avvio di sessione.)

2. La banca mock

Un assistente bancario ha bisogno di una banca. `lumera-fastapi-server/` è un'app FastAPI di ~70 righe che fa da core banking per il resto del corso:

- `GET /health` — controllo pubblico di liveness
- `GET /v1/balance?account_type=...` — saldi per conto, protetti da bearer-token
- `POST /v1/appointments` — registra una prenotazione, restituisce un riferimento come `LUM-0001`, protetto da token

È deliberatamente minuscola — abbastanza piccola da leggere in classe, e abbastanza piccola da **uccidere di proposito** quando testiamo il fallimento. Dalla cartella di questa lezione:

```
make setup          # one-time: builds both venvs (server + assistant, with rasa-
                    # pro + requests)
make run-fastapi-server # terminal 1 - the bank, on :8000
```

Verifica che risponda — nota il token, e che uno sbagliato venga rifiutato:

```
curl http://localhost:8000/health
# {"status":"ok"}
curl -H "Authorization: Bearer test_token" "http://localhost:8000/v1/balance?
account_type=savings"
# {"account_type":"savings","balance":2087.5}
curl -H "Authorization: Bearer wrong" "http://localhost:8000/v1/balance?
account_type=savings"
# {"detail":"Invalid token"}
```

3. Action 1 — `action_fetch_balance` : gli event in uscita

Lo stub del saldo muore per primo. Crea `actions/fetch_balance.py` :

```

import os
from typing import Any, Dict, List, Text

import requests
from rasa_sdk import Action, Tracker
from rasa_sdk.events import SlotSet
from rasa_sdk.executor import CollectingDispatcher

class ActionFetchBalance(Action):
    def name(self) -> Text:
        return "action_fetch_balance"

    def run(
        self,
        dispatcher: CollectingDispatcher,
        tracker: Tracker,
        domain: Dict[Text, Any],
    ) -> List[Dict[Text, Any]]:
        api_base = os.environ.get("CORE_BANKING_URL")
        token = os.environ.get("CORE_BANKING_TOKEN")
        account_type = tracker.get_slot("account_type")

        try:
            response = requests.get(
                f"{api_base}/v1/balance",
                params={"account_type": account_type},
                headers={"Authorization": f"Bearer {token}"},
                timeout=5,
            )
            response.raise_for_status()
            balance = response.json()["balance"]
            return [
                SlotSet("current_balance", balance),
                SlotSet("balance_fetch_ok", True),
            ]
        except (requests.RequestException, KeyError, TypeError, ValueError):
            return [
                SlotSet("current_balance", None),
                SlotSet("balance_fetch_ok", False),
            ]

```

Leggilo riga per riga — ogni riga è una decisione di policy:

- **L'URL e il token provengono dall'environment.** Nessuna credenziale compare nel sorgente, nello YAML, negli artefatti del modello, o nei prompt. L'LLM non li vede mai.
- **timeout=5 non è decorazione.** Un backend bloccato diventa un fallimento *progettato* in cinque secondi invece di un'attesa silenziosa.
- **L'action non decide cosa dire.** Restituisce i fatti come event: il saldo quando disponibile, e se il recupero è andato a buon fine (`balance_fetch_ok` , un flag di stato su cui il flow ramificherà).

- Il percorso di fallimento azzera `current_balance` — un saldo obsoleto proveniente da una chiamata precedente andata a buon fine non deve sopravvivere a un fallimento successivo.
- Ricorda la mapping `controlled` del Day 7: un event `SlotSet` da una custom action è esattamente lo scrittore che quella mapping consente. L'LLM ancora non può inventare un saldo; ora *la banca* lo fornisce.

Tre tocchi di YAML lo collegano. Il domain (`domain/check_balance.yml`) guadagna lo slot di stato, la risposta di fallimento e — nuovo requisito — la **registrazione** dell'action:

```
slots:
  # ... account_type and current_balance as on Day 7 ...
  balance_fetch_ok:
    type: bool
    mappings:
      - type: controlled

responses:
  # ... existing responses ...
  utter_balance_service_down:
    - text: "We're sorry, the balance service is unavailable right now. Please try again later."

actions:
  - action_fetch_balance
```

Una stringa, tre posti: il valore di `name()` , la voce sotto `actions:` , e lo step nel flow. Devono corrispondere esattamente.

E il flow (`data/flows.yml`) sostituisce la bugia con l'action più una ramificazione sul suo flag di stato:

```
check_balance:
  name: check account balance
  description: Look up the current balance of one of the customer's Banca Lumera accounts.
  steps:
    - collect: account_type
      description: "the account to check: checking or savings"
    - action: action_fetch_balance
      next:
        - if: slots.balance_fetch_ok
          then: show_balance
        - else:
          - action: utter_balance_service_down
            next: END
    - action: utter_current_balance
      id: show_balance
```

L'action *potrebbe* pronunciare le scuse essa stessa — ma questo seppellirebbe un percorso infelice dentro Python. Il fallimento è una decisione conversazionale, quindi vive nel flow, revisionabile e

visibile nel diagramma dell'Inspector. Questo è "la logica nei flow, il lavoro nelle action" in una schermata di YAML.

Eseguito — e nota ciò di cui non abbiamo avuto bisogno

Dove *viene eseguito* questo Python? Guarda `endpoints.yml`, invariato dal Day 7:

```
action_endpoint:  
  actions_module: "actions"
```

Questo è il cablaggio **in-process**: Rasa importa il package `actions` ed esegue il tuo codice dentro il processo stesso dell'assistente. Nessun server aggiuntivo, nessun terminale aggiuntivo — per lo sviluppo, questo è tutto ciò che serve. (L'altro cablaggio ha il suo momento nel §6.)

```
rasa train  
rasa inspect      # keep the mock bank running in terminal 1
```

Chiedi **"what's my savings balance?"**:

bot: Your savings account balance is €2087.5.

Il numero ora proviene dall'API — chiedi il conto checking e ottieni un numero *diverso* (€1250.0), cosa che lo stub non avrebbe mai potuto fare. Nel pannello degli slot dell'Inspector, osserva `current_balance` e `balance_fetch_ok` comparire insieme subito dopo l'esecuzione dello step dell'action.

4. Action 2 — `action_confirm_appointment` : il tracker

Seconda bugia: la prenotazione che non è mai avvenuta. Il compito di questa action è per lo più *leggere* — tutto ciò che la conversazione ha raccolto, estratto di nuovo dal tracker e spedito alla banca. Crea `actions/confirm_appointment.py`:

```

import os
from typing import Any, Dict, List, Text

import requests
from rasa_sdk import Action, Tracker
from rasa_sdk.events import SlotSet
from rasa_sdk.executor import CollectingDispatcher

class ActionConfirmAppointment(Action):
    def name(self) -> Text:
        return "action_confirm_appointment"

    def run(
        self,
        dispatcher: CollectingDispatcher,
        tracker: Tracker,
        domain: Dict[Text, Any],
    ) -> List[Dict[Text, Any]]:
        api_base = os.environ.get("CORE_BANKING_URL")
        token = os.environ.get("CORE_BANKING_TOKEN")

        # Everything the conversation collected, read back from the tracker.
        payload = {
            "customer_id": tracker.sender_id,
            "branch_city": tracker.get_slot("branch_city"),
            "topic": tracker.get_slot("appointment_topic"),
            "preferred_time": tracker.get_slot("preferred_time"),
        }

        try:
            response = requests.post(
                f"{api_base}/v1/appointments",
                json=payload,
                headers={"Authorization": f"Bearer {token}"},
                timeout=5,
            )
            response.raise_for_status()
            reference = response.json()["reference"]
            return [
                SlotSet("booking_reference", reference),
                SlotSet("booking_ok", True),
            ]
        except (requests.RequestException, KeyError, TypeError, ValueError):
            return [
                SlotSet("booking_reference", None),
                SlotSet("booking_ok", False),
            ]

```

Il tour del tracker, in tre letture:

- `tracker.get_slot("...")` — gli slot che il flow ha raccolto nel Day 7, una chiamata ciascuno. Il flow li ha raccolti; l'action li consuma.
- `tracker.sender_id` — l'identificatore stabile della conversazione, qui usato anche come customer id (mock). In produzione è qui che emerge lo user id autenticato del tuo canale.
- `tracker.latest_message` — non usato nel payload, ma stampalo una volta in classe: contiene il testo grezzo dell'ultimo messaggio dell'utente (il "yes please, book it") più i metadati strutturati che la pipeline vi ha allegato. Quando un'action ha bisogno di sapere *come* qualcosa è stato detto, è qui che si guarda.

Il cablaggio, gli stessi tre tocchi — mostrati per intero, come con `check_balance` sopra.

`domain/book_appointment.yml` guadagna i due slot controlled, la risposta di fallimento `utter_booking_failed`, e la registrazione dell'action; e l'esistente `utter_appointment_booked` ora interpola un artefatto reale dell'integrazione:

```
slots:
  # ... branch_city, appointment_topic, preferred_time, appointment_confirmed as on Day
  7 ...
  booking_ok:
    type: bool
    mappings:
      - type: controlled
  booking_reference:
    type: text
    mappings:
      - type: controlled

responses:
  # ... existing appointment responses ...
  utter_appointment_booked:      # Day 7's response, now carrying the reference
    - text: "Done — your appointment is booked, reference {booking_reference}. You'll
receive a confirmation shortly."
  utter_booking_failed:
    - text: "I couldn't reach the booking system, so nothing was booked. Please try
again in a few minutes."

actions:
  - action_confirm_appointment
```

Nel flow, il ramo di conferma smette di essere teatro conversazionale — su "yes" esegue l'action e ramifica sull'esito:


```

- collect: appointment_confirmed
description: whether the customer confirms the recap of the appointment
next:
  - if: slots.appointment_confirmed
    then:
      - action: action_confirm_appointment
        next:
          - if: slots.booking_ok
            then:
              - action: utter_appointment_booked
                next: END
          - else:
            - action: utter_booking_failed
              next: END
    - else:
      # ... the Day 7 decline branch, unchanged ...

```

Riaddestra ed esegui l'intera prenotazione. Il recap e la conferma sono esattamente come ieri — ma la riga di chiusura ora porta il riferimento della banca:

bot: Done — your appointment is booked, reference LUM-0002. You'll receive a confirmation shortly.

Quel riferimento è stato coniato dall'API mock, che ora ha l'appuntamento registrato. La conversazione ha prodotto un effetto durevole in un altro sistema — questo è il confine di integrazione, attraversato.

5. Action 3 — `action_send_statement_link` : il dispatcher

Terzo volto: un'action il cui unico compito è l'*output*. Nuova piccola capacità — il cliente vuole l'estratto conto, e la risposta è un **deeplink costruito a partire dai valori degli slot**. Crea `actions/send_statement_link.py` :

```

from typing import Any, Dict, List, Text

from rasa_sdk import Action, Tracker
from rasa_sdk.executor import CollectingDispatcher

class ActionSendStatementLink(Action):
    def name(self) -> Text:
        return "action_send_statement_link"

    def run(
        self,
        dispatcher: CollectingDispatcher,
        tracker: Tracker,
        domain: Dict[Text, Any],
    ) -> List[Dict[Text, Any]]:
        account_type = tracker.get_slot("account_type")
        url = f"https://banca-lumera.example/statements/{account_type}"

        dispatcher.utter_message(
            text=f"Here is the statement of your {account_type} account:"
        )
        dispatcher.utter_message(
            json_message={
                "type": "deeplink",
                "url": url,
                "label": f"Download {account_type} statement (PDF)",
            }
        )
        return []

```

`dispatcher.utter_message(...)` è il multiutensile dell'output; la sua keyword decide la forma. `text=` invia parole semplici; `response="utter_x"` innesca una risposta del domain per nome (mantenendo la formulazione nello YAML anche quando è Python a decidere il momento); `buttons=` allega delle scelte; e `json_message=` invia un **payload personalizzato arbitrario** — il canale lo trasmette intatto affinché l'applicazione client lo renderizzi. Quest'ultimo è il modo in cui cose strutturate — deeplink, card, widget — viaggiano da un'action a un frontend. Nota che questa action restituisce `[]`: non ha cambiato nulla nello stato della conversazione; ha solo *detto* delle cose.

Collegalo con un quarto flow (in `data/flows.yml`) e un file di domain di due righe (`domain/download_statement.yml`) che si limita a registrare l'action:

```

download_statement:
  name: download account statement
  description: Send the customer a link to download the statement of one of their
accounts.
  steps:
    - collect: account_type
      description: "the account whose statement the customer wants: checking or
savings"
    - action: action_send_statement_link

```

E `domain/download_statement.yml` per intero — nessun nuovo slot o risposta, registra soltanto l'action:

```

version: "3.1"

actions:
  - action_send_statement_link

```

Nessun nuovo slot: `download_statement` raccoglie lo stesso `account_type` che usa `check_balance` — gli slot appartengono alla conversazione, non a un flow. (Se hai controllato un saldo prima nella sessione, il flow dell'estratto conto non lo chiederà nemmeno.)

Riaddestra e prova **"I need the statement of my savings account"** — attraverso l'API REST la risposta torna come un messaggio di testo più un payload `custom`:

```

{"text": "Here is the statement of your savings account:"}
{"custom": {"type": "deeplink",
  "url": "https://banca-lumera.example/statements/savings",
  "label": "Download savings statement (PDF)"}}

```

Un deeplink parametrizzato, assemblato a partire dallo stato della conversazione, consegnato come JSON leggibile da una macchina: esattamente l'handoff che un frontend web o app consuma.

6. Il secondo cablaggio: un action server esterno

Il modulo in-process ci ha servito tre action senza alcuna cerimonia. Ora vedi l'altra forma — perché in produzione il codice delle action di solito **non** vive dentro il processo dell'assistente. Due terminali:

```
make run-actions-server    # terminal 2 - rasa run actions, an HTTP server on :5055
```

e cambia `endpoints.yml` da modulo a URL:

```

action_endpoint:
  url: "http://localhost:5055/webhook"

```

Riavvia l'assistente (`make run-rasa`) ed esegui un qualsiasi controllo del saldo — comportamento identico, anatomia diversa: ogni step di action è ora un round-trip HTTP verso un processo separato.

Perché dovresti volerlo?

- **Isolamento** — il codice delle action va in crash, si aggiorna e scala senza toccare l'assistente; le dipendenze pesanti restano fuori dal processo Rasa.
- **Separazione delle credenziali** — provalo: il terminale dell'assistente non ha più bisogno di *alcun* `CORE_BANKING_TOKEN` . Solo l'action server detiene le credenziali della banca; il processo rivolto al modello non può letteralmente far trapelare ciò che non ha.
- **Confine di team** — l'action server è un semplice servizio Python che il tuo team di backend può possedere, testare e distribuire come qualsiasi altro.

Entrambi i cablaggi sono di prima classe; la scelta è una questione di pragmatismo per progetto. **Per questo corso ora torniamo al modulo in-process** — un terminale in meno ogni sessione, e nulla nelle lezioni a venire dipende dal confine. Ripristina:

```
action_endpoint:  
actions_module: "actions"
```

Quando il corso raggiungerà gli argomenti di produzione, il server esterno tornerà come la forma corretta per il confine di integrazione.

7. Il teatro del fallimento

Due epiloghi, messi in scena deliberatamente. Tieni l'assistente in esecuzione.

Il fallimento progettato. Ferma la banca mock (Ctrl-C nel terminale 1) e chiedi un saldo:

bot: We're sorry, the balance service is unavailable right now. Please try again later.

L' `except` dell'action ha catturato l'errore di connessione, `balance_fetch_ok` è tornato `false` , e il *nostro* ramo del flow ha prodotto una risposta brandizzata, specifica, recuperabile. Prova anche la prenotazione — ha il suo fallimento progettato:

bot: I couldn't reach the booking system, so nothing was booked. Please try again in a few minutes.

("Nothing was booked" non è un riempitivo — l'action lo garantisce: nessun riferimento, nessun flag di successo, nessuna falsa promessa.)

Il fallimento non gestito. Ora saboti l'action — aggiungi un `raise RuntimeError("boom")` come prima riga di `action_fetch_balance.run()` , riavvia, e chiedi di nuovo:

bot: Sorry, I am having trouble with that. Please try again in a few minutes. **bot:** Okay, stopping pattern_collect_information.

L'eccezione ha attraversato il confine dell'SDK, Rasa ha annullato il flow attivo, e un pattern di riparazione integrato (`pattern_internal_error`) ha prodotto delle scuse generiche. Confronta le due trascrizioni: stesso backend rotto, ma una risposta è la tua e una è l'ultima risorsa del framework. Il `try / except` e lo slot con il flag di stato sono la differenza. **Rimuovi la riga di sabotaggio prima di procedere.**

8. Dove siamo arrivati

```
day-08-custom-actions-and-integration/
├─ Makefile                      # process runner (bank / action server / rasa)
├─ lumera-fastapi-server/        # the mock core-banking API
├─ lumera-assistant/
│   └─ actions/
│       ├── fetch_balance.py      # events out (SlotSet + status flag)
│       ├── confirm_appointment.py # tracker in (slots, sender_id → POST)
│       └─ send_statement_link.py # dispatcher out (text + json_message deeplink)
├─ data/flows.yml                # check_balance branches; booking lands; +
download_statement
├─ domain/                      # + controlled result/status slots, + actions:
registrations
├─ endpoints.yml                # in-process module (external URL demoed, reverted)
```

La divisione del lavoro, ora in codice funzionante:

- Il **domain** dichiara cosa può esistere: input compilabili dall'utente, fatti controlled del backend, risposte, action registrate.
- Il **flow** possiede la logica conversazionale: raccogli, esegui l'action, ramifica sul flag di stato.
- L'**action** svolge il lavoro: legge le credenziali dall'env, chiama il backend con un timeout, restituisce event — oppure parla attraverso il dispatcher.
- **endpoints.yml** sceglie il confine di processo: in-process per la comodità dello sviluppo, URL esterno per l'isolamento e la separazione delle credenziali.

Le due bugie del Day 7 non ci sono più: il saldo viene recuperato, la prenotazione va a buon fine e restituisce un riferimento. Ciò che l'assistente *dice* è ancora interamente YAML — Python ha sempre e solo fornito fatti e payload. Quella disciplina è ciò che manterrà questo codebase revisionabile man mano che cresce.